LOCAL, PER-PROMPT MODEL ROUTING

Calibra

The product guide for automatic model selection in Claude Code and Codex

Calibra intercepts each prompt in a local proxy, estimates the work tier, rewrites the model field, and forwards the request to the configured upstream AI server.

LIGHT
Recall and mechanical edits

MID
Bounded implementation

DEEP
One-system judgment

ULTRA
Multi-system programs

Product guide | Version 1.0.10 | July 2026

01 / PRODUCT

What is Calibra?

A small control plane at the exact point where coding-agent requests leave the workstation.

The problem

A fixed premium model is reliable but expensive. A fixed small model is fast and economical but can under-serve architecture, diagnosis, or multi-system work. Manual switching interrupts flow and is easy to forget.

The Calibra answer

Route every prompt automatically. The request stays compatible with the existing coding agent: Calibra changes only the request model and forwards the rest through the configured upstream path.

Who it helps

Teams using Claude Code or Codex behind enterprise AI infrastructure, especially when they want predictable tier policy, local routing logic, per-environment controls, and no prompt-by-prompt model menu.

Value	How Calibra creates it	How to measure it
Lower avoidable inference spend	Light and mid work can be mapped to less expensive models.	Compare actual routed-model cost against a fixed-model baseline.
Less context switching	Model choice happens in-flight without an agent restart or manual selector.	Track manual model overrides and workflow interruption.
Safer complex-work handling	Deep-intent floors, multi-system rules, and a cost-sensitive ML policy reduce severe under-routing.	Review under-routes, rework, and escalation rates by tier.
Policy transparency	Four named tiers map to editable model IDs in a local JSON file.	Audit tier distribution, model mapping, and exception cases.

02 / RUNTIME

How it works

A transparent five-step request path. Classification happens locally before the configured upstream call.

1 Prompt

Claude Code or Codex creates an API request.

2 Intercept

A loopback HTTP proxy reads the newest user prompt.

3 Classify

Heuristic or rule-first MiniLM assigns a tier.

4 Rewrite

The request model field is replaced from tier config.

5 Forward

The request continues to the enterprise/upstream AI server.

Tier	Default model	Typical signal	Example
LIGHT	Haiku	Social, recall, typo, one mechanical edit	Rename this variable
MID	Sonnet	Concrete work in one bounded component	Implement pagination
DEEP	Opus	Synthesis or judgment in one system	Audit the auth design
ULTRA	Opus	Multiple named systems or org-wide program	Plan auth + payments migration

The model mapping is editable. With the default file, deep and ultra use the same model; the tiers still preserve policy meaning but do not create a model-price difference until configured differently.

03 / DECISION ENGINES

Heuristic by default, ML by choice

Both paths are total and fail-soft: a valid tier is returned even when optional ML assets are unavailable.

Heuristic engine

Sequence
- Correct supported EN/TR typos when dictionaries exist.
- Apply ordered fast exits for greetings, trivial edits, slash commands, and short prompts.
- Score five axes: length, intent, scope, domain, and structure.
- Enforce intent floors and map the score to light/mid/deep/ultra.

Best when deterministic behavior, minimal dependencies, and easy inspection matter most.

Rule-first ML engine

Sequence
- Commit only high-precision rules for bright-line cases.
- Defer ambiguous prompts to a local MiniLM-L6 ONNX encoder.
- Mean-pool 384-dimensional embeddings, add lexical features, and run an ordinal head.
- Choose the tier with a cost-sensitive decision policy.
- Fall back silently to heuristics on missing model, timeout, or error.

Best when ambiguous boundaries justify the local model footprint.

Repository benchmark	Accuracy	Severe under-route	Reading
Dev - 500 rows	93.4%	0.00%	Tuned set; not an independent headline.
Calibra test - 600 rows	90.33%	0.67%	Independent report set after lexicon fixes.
Adversarial - 600 rows	88.67%	0.17%	Boundary-stress test; lower cost than baseline.
Independent labeler runs	76%-88%	Varies	Shows sensitivity to tier rubric, writing style, and typos.

04 / BUILD STORY

How Calibra is created

A Node.js proxy product with deterministic policy, an optional local embedding classifier, and reproducible evaluation tooling.

Agent integration	Claude hooks + slash command | Codex config patch + local service
Proxy layer	HTTP request parsing, model rewrite, sanitization, upstream forwarding
Routing policy	Four-tier rubric, per-environment flags, editable model mapping
Decision engines	Five-axis heuristic | rule-first MiniLM ONNX cascade
Validation assets	Golden/rule tests, labeled JSONL sets, cost matrices, threshold tools

Core repository map

src/saka-proxy.js - Claude request proxy
src/codex-proxy.js - Codex proxy and commands
src/ml/ - tokenizer, rules, ONNX runtime, ordinal head
scripts/install.js - installation and integration
tools/ - training, evaluation, thresholds
docs/ - baseline, results, Draw.io flows

Development loop

1. Define the four-tier rubric.
2. Encode high-precision deterministic cases.
3. Train/evaluate the ambiguous residual.
4. Optimize routing cost, especially severe under-routing.
5. Test against dev, independent, adversarial, bilingual, and typo-heavy sets.
6. Ship with fail-soft fallbacks and user-owned model mappings.

Licensing: the package declares MIT. The MiniLM base model is documented as Apache-2.0 in docs/RESULTS.md.

05 / INSTALLATION

Install in one command, then verify the environment

The installer writes local integration files, preserves user-owned tier configuration, and prepares optional ML assets.

1. Prerequisites

- Node.js 18 or newer
- Claude Code CLI and/or Codex CLI
- For Claude Code: an enterprise wrapper/upstream path that sets the expected proxy environment
- Access to the configured upstream AI server

2. Recommended install

```
npx calibra install
```

Alternative:

```
npm install -g calibra
```

The package requires Node.js 18+ and runs its installer as postinstall.

3. Configure and verify

```
export
CALIBRA_REMOTE_HOST="gateway.example.com"
```

Claude Code: `/calibra status`
Codex: `status calibra`

Review tier mapping at:
`~/.claude-corp/calibra/calibra-models.json`

Installer action	Why it matters	Safety behavior
Copies proxy, hooks, command, ML runtime files	Puts the routing path and controls where the CLIs can use them.	Existing user model and ML JSON files are not overwritten.
Patches Claude settings and detected Codex config	Registers hooks and points Codex at a loopback provider.	Codex config is backed up once for uninstall; patch is idempotent.
Registers Codex background proxy	Keeps the fixed local endpoint available across sessions.	Automatic registration is implemented for macOS and Windows; other platforms print a manual start command.
Downloads ~22 MB ONNX model + runtime deps	Enables optional local MiniLM routing.	SHA-256 verified and atomically moved; failure leaves heuristic routing available.

Lifecycle: upgrade with npx calibra upgrade. Remove installed files, hooks, and registered entries with npx calibra uninstall.

06 / DAILY USE

How to use Calibra

Routing is silent by default. The visible note confirms the model and tier selected for the current prompt.

Claude Code commands

`/calibra status` - show state
`/calibra on` - enable routing
`/calibra off` - keep current/original model
`/calibra toggle` - flip state
`/calibra ml on` - use rule-first ML
`/calibra ml off` - use heuristic
`/calibra rules` - alias for ML off

Natural language also works: `enable calibra` / `disable calibra`.

Codex phrases

Codex has no Calibra slash command. Type plain text:

```
status calibra
enable calibra
disable calibra
turn on calibra
turn off calibra
calibra ml on
calibra ml off
calibra rules
```

The routing note appears at the top of the response.

Prompt	Expected tier	Why
Rename this variable	LIGHT	Single mechanical edit; no new logic.
Implement pagination in the users endpoint	MID	Concrete work in one bounded component.
Audit the authentication architecture and propose hardening	DEEP	One-system judgment and security analysis.
Plan an end-to-end migration across auth, payments, and CRM	ULTRA	Multiple named systems plus broad program scope.

State is separate by environment. Enabling, disabling, or switching the engine in Claude Code does not change Codex, and vice versa.

07 / ASSESSMENT

Pros, cons, and operational risks

Calibra is strongest as a focused, local coding-agent router. Its narrowness is both the product advantage and the main constraint.

Pros

- Transparent per-prompt routing with no manual model picker.
- Local classification path; no separate routing SaaS is required.
- Deterministic default plus an optional semantic classifier.
- Fail-soft behavior preserves a usable tier on ML failure.
- Editable tier-to-model policy is not overwritten on upgrade.
- Separate controls for Claude Code and Codex.
- English/Turkish typo handling and bilingual rules.
- Reproducible tests, labeled sets, and honest error analysis.

Cons and constraints

- Requires a local proxy and integration changes to CLI configuration.
- Claude use assumes an enterprise wrapper/upstream arrangement.
- Any classifier can over-route cost or under-route quality.
- Repo benchmarks measure tier labels, not task success or savings.
- ML adds a native runtime, dictionaries, and a ~22 MB model.
- Deep and ultra map to the same default model unless customized.
- Per-environment state can surprise users expecting one global switch.
- Linux Codex autostart is manual in the current installer code.

Risk	Control	Owner metric
Complex prompt under-routed	Keep severe-under costs high, sample decisions, preserve an off switch.	Under-route/rework rate
Simple prompt over-routed	Review light/mid precision and customize tier models.	Cost per accepted task
Proxy/config disruption	Use backups, idempotent install, health checks, and uninstall rehearsal.	Proxy failures / 502s
Benchmark drift	Label production prompts and re-evaluate by language, typo rate, and team.	Agreement by segment

08 / MARKET CONTEXT

Similar products and how they differ

A July 2026 snapshot. The nearest alternatives solve adjacent routing problems but differ in scope, hosting, and integration style.

Product	Primary fit	Routing basis	Execution / integration	Best advantage / trade-off
Calibra	Claude Code and Codex in enterprise setups	Four complexity tiers; heuristic or local MiniLM cascade	Local Node proxy; npx installer patches agent integrations	Very focused and locally controlled / narrower ecosystem and config coupling
RouteLLM	Application and research routing	Learned routers, typically strong vs weak model with threshold	Open-source Python controller or OpenAI-compatible server	Extensible and benchmark-oriented / requires calibration and app integration
OpenRouter Auto	General hosted multi-model inference	Task-aware managed routing across a curated model pool	Hosted API; request model is openrouter/auto	Fastest broad-model adoption / third-party routing, pricing, and data boundary
Not Diamond	Apps and long-running coding agents	Pre-trained or custom router; quality, cost, or latency objective	Hosted API with Python, TypeScript, and REST	Custom router and prompt optimization / account, API key, managed service
LiteLLM	Organization-wide AI gateway	Provider/deployment routing, load balancing, retries, fallback, policy	Self-hosted gateway or Python SDK	Broad provider, spend, guardrail, and ops layer / not the same coding-task tier classifier by default

Choose Calibra when

You want agent-specific, local, transparent routing for Claude Code/Codex and control the upstream model mapping.

Choose a router framework when

You need to train, evaluate, and serve routing logic inside a broader application stack.

Choose a gateway/service when

You need many providers, centralized governance, managed model pools, spend controls, or hosted routing.

09 / ADOPTION

A practical adoption path

Treat routing as a measurable policy change. Start small, observe decisions, and expand only after cost and quality are both visible.

1. Baseline

Record fixed-model cost, latency, rework, and task acceptance for representative prompts.

→

2. Shadow

Classify prompts and log suggested tiers without changing the production model mapping.

→

3. Pilot

Enable routing for a small team; keep /calibra off or disable calibra as the immediate escape hatch.

→

4. Tune

Adjust tier models and review under-routes by language, task type, and prompt style.

→

5. Scale

Publish policy, monitor drift, rehearse upgrade/uninstall, and review upstream/provider changes.

Sources and comparison basis

Comparison claims are based on public product documentation accessed 22 July 2026. Product features and pricing can change. Calibra claims are based on the checked-out repository, not an external marketing page.

Local product sources	README.md; package.json; scripts/install.js; src/saka-proxy.js; src/codex-proxy.js; src/ml/*; docs/BASELINE.md;...
RouteLLM	https://github.com/lm-sys/RouteLLM
OpenRouter Auto Router	https://openrouter.ai/docs/guides/routing/routers/auto-router
Not Diamond model routing	https://docs.notdiamond.ai/docs/what-is-model-routing
LiteLLM documentation	https://docs.litellm.ai/
Atmosware logo	https://www.atmosware.com.tr/images/corp-logos/atmosware_logo.png